

박창준 협업일지 요약

중급프로젝트 RFP RAG · 기록 기간 2026-07-13 ~ 2026-08-03

역할 요약

박창준님은 프로젝트의 데이터 파이프라인 담당자로서 원본 EDA, HWP/PDF 구조 전처리, Naive-Advanced 청킹, OpenAI 임베딩, Chroma-BM25 인덱싱, 이미지 OCR 가능성 조사와 최종 품질 검증을 수행했다. 단순히 코드를 작성하는 데서 멈추지 않고 입력 SHA-256, 문서·청크 수, 토큰 상한, metadata, 실행시간과 공유 권한을 계약으로 남겨 다음 검색 담당자가 같은 결과를 재현할 수 있게 했다. 표 구조와 오버랩처럼 설정값만으로 확인할 수 없는 문제는 원본과 산출물을 직접 대조해 수정했으며, 마지막에는 전체 과정을 발표 자료와 파트 보고서로 정리했다.

핵심 기여

1. 원본 EDA와 데이터 기준선 확정

- 원본 HWP 96개와 PDF 4개, 총 100개를 기준으로 EDA를 수행했다.
- CSV 100행과 원본 파일을 매칭하고 SHA-256 중복 2쌍을 찾아, 원본은 보존하되 검색 대상은 98개 고유 문서로 확정했다.
- CSV 본문을 검색 원문으로 사용하지 않고, 공고번호·기관·금액·날짜를 검증·보강하는 metadata 자료로만 사용하는 원칙을 정리했다.
- HWP/PDF 추출 실패, 짧은 본문, 목차와 제목 구조, 표·이미지 신호를 검토 후보로 분리했다.
- EDA → 전처리 → 검증 → 청킹을 분리해 팀원이 각 단계의 목적과 산출물을 이해할 수 있도록 코드와 문서를 정리했다.

2. rhwp 기반 Naive 구조 처리와 512/102 청킹

- HWP의 일반 텍스트·표·이미지 구조를 rhwp로 읽고, 이미지는 JSON 변환 대신 `to_ir()` 구조에서 확인하도록 정리했다.
- 표는 검색 가능한 텍스트 형태로 변환하고, 이미지 바이너리·Base64·placeholder는 임베딩하지 않는 기준을 세웠다.
- Naive Recursive 512토큰·102토큰 오버랩 청킹을 구현하고 PR을 통해 main에 병합했다.
- 청크 압축 파일에 고정 `chunk_id`, `source_id`, 원본 SHA, 문서·위치·전략 metadata를 포함해 중단 후 재실행과 결과 대조가 가능하도록 했다.

3. GCP Naive 임베딩과 Chroma 구축

- GCP 개인 경로에 저장소와 uv 환경을 재현하고 전체 테스트를 통과시킨 뒤 3개 청크 smoke test를 먼저 수행했다.
- `text-embedding-3-small`, 1,536차원, Collection `ai11_policy`, 경로 `/home/data/chroma`를 인덱스 계약으로 확정했다.

- `Chroma.from_documents()` 한 번으로 처리하지 않고 OpenAI 임베딩 호출과 Chroma upsert를 분리해 두 구간 시간을 각각 측정했다.
- 기존 smoke 3개를 건너뛰고 31,336개를 추가해 최종 31,339개를 저장했다.

항목	결과
최종 체크·벡터	31,339개
임베딩 API 누적	239.754878초
Chroma 저장 누적	138.617121초
전체 누적	383.368609초, 약 6분 23초
저장 용량	약 459MB

- 저장 후 Collection count와 실제 `similarity_search(k=3)`를 확인해 임베딩 → 저장 → 검색의 전체 흐름을 검증했다.
- `/home/data/chroma`와 보고서 경로를 `teamshare` 그룹으로 관리하고, 팀원이 같은 모델·경로·Collection을 사용하도록 인계했다.

4. Advanced 구조 전처리

- Naive 코드와 분리된 Advanced 전처리 파이프라인을 구현하고 함수별 한국어 docstring과 복잡한 분기 설명을 추가했다.
- 일반 텍스트와 표를 전처리 단계부터 분리했다. 일반 텍스트만 KSS·Kiwi 대상이 되고, 표는 HTML과 Markdown을 모두 저장하되 Markdown을 Dense 검색 본문으로 사용하도록 했다.
- PDF는 실제 1-based page, HWP/HWPX는 `section_idx + para_idx`로 위치를 기록했다. HWP에 없는 페이지 번호를 추정하지 않았다.
- GCP 전체 실행으로 98개 문서, 94,378개 블록, 표 12,487개, 이미지 886개를 생성했다.
- GCP의 FreeType ABI 오류는 시스템 전체를 바꾸지 않고 전처리 wrapper 실행에만 호환 라이브러리를 적용해 해결했다.

5. PDF·HWP 표 구조 개선

- 팀에서 제안한 `fitz` 방식과 기존 `pdfplumber`를 고려해 PDF로 실측 비교해 `pdfplumber` 유지로 결론 났다.
- PDF 표 셀 좌표로 격자를 재구성해 `rowspan·colspan`을 복원했다.
- HTML에는 병합 구조를 보존하고, Markdown에는 병합 값을 반복 채워 각 행이 검색 가능한 형태가 되도록 역할을 분리했다.
- 표 691개를 대조해 원문에 없는 값이 생성된 사례가 0건임을 확인했다.
- 표가 512토큰 경계에서 잘리지 않도록 8,191토큰 상한 안에서는 전체 표를 하나의 청크로 유지했다.
- `>`, `<`, `&` 같은 원문 기호와 빈 셀을 보존하고, 원문에 없는 병합 주석·내부 참조 문자열이 임베딩에 들어가지 않도록 정리했다.

6. Advanced 512·1024·Semantic 청킹과 BM25

- HWP 일반 텍스트를 지나치게 짧은 문단 단위로 두지 않고 `section_path` 안에서 문장을 목표 토큰까지 묶는 방식으로 개선했다.
- KSS 입력을 400자 이하로 나누고 3개 worker를 사용해 예상 13~20시간 작업을 36.8분으로 단축했다.
- Kiwi 형태소 분석 결과를 일반 텍스트 BM25 토큰으로 저장하고, 표 BM25는 팀 결정에 따라 검색용 평문 또는 정책에 맞는 별도 필드로 관리했다.
- 내부 표·이미지 참조 ID가 BM25 토큰으로 들어가던 문제를 찾아 검색 본문에서 제거하고 metadata로 옮겼다.

짧은 청크 편향 해결

문단별 청킹에서는 텍스트 청크 41,830개의 평균 길이가 45.7토큰이었고, 질의 8개 × k=5 결과에서 표는 1/40건만 나타났다. `section_path` 안에서 문장을 패킹하자 1~50토큰 청크가 사라지고 표가 28/40건으로 증가했다. 표가 너무 커서 검색되지 않는 것이 아니라, 짧은 텍스트 청크가 상위를 점유한 것이 원인이었다.

오버랩 실행작 수정

512/51 설정은 존재했지만 RFP 문장 중앙값이 96토큰이라, 문장 전체만 옮기는 기존 방식으로는 51토큰 오버랩을 만들 수 없었다. 문장 경계로 채울 수 없을 때 UTF-8 안전한 토큰 경계 fallback을 적용했다.

지표	수정 전	수정 후
오버랩 기록 있음	28.7%	99.7%
원문이 실제로 겹침	26.0%	97.1%

- 수정 후 metadata에 실제 오버랩 토큰 수와 문장 수를 노출해 Vector DB만 보고도 확인할 수 있게 했다.

세 조건 구축

조건	인덱싱 벡터	특징
512/51	18,258	짧은 응답 구간, 10% 오버랩
1024/102	15,483	더 긴 문맥, 10% 오버랩
Semantic p83	15,985	이웃 문장 거리 임계값 0.706, 의미 경계 1,433개

- 시멘틱 청킹은 문장 21,455개를 임베딩하고 이웃 거리 20,698개를 계산해 p83 임계값을 선택했다.
- 문장 임베딩을 재사용 파일로 저장해 임계값 실험 때 API를 반복 호출하지 않도록 했다.
- 리팩터 후에도 기존 1024 산출물 386개가 ID·본문·토큰·BM25·오버랩에서 완전히 일치하는지 확인했다.

7. 이미지 OCR 조사와 자원 관리

- HWP/PDF 이미지 1,006개를 전수 조사하고 아이콘·작은 그래픽·SHA 중복을 제거해 고유한 큰 그림 424개를 추렸다.
- 이미지로 삽입된 사업기간·예산·사용자 수 표가 기존 텍스트·표 파이프라인에서 빠질 수 있음을 확인했다.
- Tesseract 표본 20장에서 한글 30자 이상 회수 70%를 확인한 뒤, VM에서 OCR 엔진 6종을 비교했다.
- EasyOCR GPU는 로컬 CPU보다 약 19.5배 빨랐고, gpt-5-mini는 핵심어 23/23·어려운 값 8/8을 회수했다.
- 실패 표본이 모델 문제가 아니라 빈 페이지·로고·배너·장식 그래픽이라는 것을 실물과 측정값으로 확인했다.
- OCR 실험으로 VM 디스크가 98%까지 찼을 때 실험용 패키지를 정리해 91%로 복구하고, 공용 GPU-RAM 작업과 충돌하지 않도록 자원 상태 점검 원칙을 남겼다.

8. 검증·문서화·발표

- 각 단계에 단위 테스트, `prek`, 입력 SHA, count, 품질 gate와 독립 검증을 적용했다.
- 보고서만 통과했다고 끝내지 않고 원본 표·연속 청크·검색 결과를 직접 열어 확인했다.
- 시멘틱 PR을 병합해 `main`에서 512·1024·Semantic 조건을 재현할 수 있게 했다.
- OCR 엔진 비교를 시각 리포트로 만들고 색상만으로 성공·실패를 구분하지 않도록 접근성을 보완했다.
- 8월 3일 EDA → 전처리 → 청킹 문제 해결 → 인덱싱 → 검증 → OCR 순서의 18장 발표 자료와 대본을 완성하고 예상 질문 10개를 준비했다.

기간별 작업 흐름

기간	주요 작업	대표 결과
7/13~7/14	원본 EDA·중복·결측·추출 기준	100개 원본 → 98개 고유 문서
7/20~7/21	Naive 청킹·임베딩·Chroma	31,339개 벡터, 실제 검색 검증
7/22~7/26	Advanced 전처리와 표 구조 복구	94,378개 블록, 표 HTML·Markdown 보존
7/27~7/30	512·1024·Semantic 청킹·인덱싱·오버랩 수정	세 조건 재현, KSS 36.8분, 오버랩 99.7%
7/29~7/31	이미지 전수 조사·OCR 엔진 비교	1,006개 조사, 처리 후보 424개, 6종 비교
8/3	보고서·발표·파트 인계	18장 발표 자료와 대본·예상 질문 완성

문제 해결에서 드러난 강점

- 1 설정값보다 실제 산출물을 검증했다. 오버랩 51이라는 값이 있어도 원문이 겹치지 않는 것을 발견해 실제 문자열 대조 검사를 추가했다.
- 2 표현·검색·표시 목적을 분리했다. Markdown은 검색, HTML은 병합 구조와 원본 표시, metadata는 위치·참조 정보로 역할을 나눴다.
- 3 재현성과 비용을 관리했다. smoke test, 고정 ID, SHA-256, 재개 가능한 upsert와 임베딩 캐시로 실패 시 API 비용과 재실행 범위를 줄였다.
- 4 공용 환경을 고려했다. GCP 권한, 디스크, RAM·GPU 상태를 점검하고 팀원이 동일 DB를 읽을 수 있게 인계했다.
- 5 실험 결과가 이상할 때 측정 기준도 의심했다. OCR의 Unicode 정규화와 표 구조 지표 문제를 찾아 잘못된 결론을 수정했다.